Step 1 — A Minimal Server With One Tool — Solution

> Back to this step · Hint 1 · Hint 2 · Solution

Story — Step 1: every later layer this project adds (real storage, a Resource, a Prompt, a real client) is worthless if the Model Context Protocol handshake itself doesn't work. This step proves that first, with the smallest possible server: one `@mcp.tool()`-decorated function, backed by an in-memory list, and nothing else to go wrong. **If not:** when Step 3's Resource misbehaves, you won't know whether the bug is in the Resource or in the connection underneath it.

After this step the server runs: `mcp dev server.py` opens the Inspector and `add_note` responds, and the test files print `OK`.

The shared files

Doc01's config and logger, in the `src/` package. This server calls no LLM, so there's no API key — only `LOG_LEVEL` and `NOTES_DB_PATH`, both optional.

Story — `exceptions.py`, `config.py`, `utils/logger.py`: settings load from one place as usual, but logging is deliberately configured to write to **stderr**, not stdout. **If not:** on the stdio transport, stdout *is* the protocol channel itself, so one stray log line sent to stdout would corrupt every message the client tries to read.

```
# src/exceptions.py
# The project's named errors. MissingConfigError: a required setting is
# missing. (EmptyNoteError is added in Step 2.)
class MissingConfigError(Exception):
    """Raised when a required setting is missing from the environment."""
    pass
```

```
# src/config.py
# Loads settings from .env into a checked Config. This server calls no LLM,
# so there is no API key — only LOG_LEVEL and NOTES_DB_PATH, both optional.
import os
from dataclasses import dataclass
from dotenv import load_dotenv
from src.exceptions import MissingConfigError
```

```
@dataclass
class Config:
    log_level: str
    notes_db_path: str

def require_env(key):
    value = os.getenv(key)
    if value is None or value == "":
        raise MissingConfigError(
            f"Required environment variable is missing: {key}"
        )
    return value

def load_config():
    load_dotenv()
    log_level = os.getenv("LOG_LEVEL", "INFO")
    notes_db_path = os.getenv("NOTES_DB_PATH", "notes.db")
    return Config(log_level=log_level, notes_db_path=notes_db_path)
```

```
# src/utils/logger.py
# get_logger(name): one logging setup for every file. It logs to STDERR on
# purpose — on the stdio transport, stdout IS the MCP protocol channel.
import logging
import os
import sys

def get_logger(name):
    logger = logging.getLogger(name)
    if not logger.handlers:
        level_name = os.getenv("LOG_LEVEL", "INFO")
        level = getattr(logging, level_name.upper(), logging.INFO)
        handler = logging.StreamHandler(sys.stderr)
        handler.setLevel(level)
        log_format = "%(asctime)s %(name)s %(levelname)s %(message)s"
        handler.setFormatter(logging.Formatter(log_format))
        logger.addHandler(handler)
        logger.setLevel(level)
    return logger
```

Also create empty `__init__.py` files in `src/` and `src/utils/`.

> Back to this step · Hint 1 · Hint 2 · Solution

Solution — one tool, an in-memory list, over stdio

Story — `server.py`: this is the smallest real MCP server — an `MCPServer` instance, one `@mcp.tool()`-decorated function, and `mcp.run(transport="stdio")` to actually start listening. The note goes in a plain in-memory list, which means it forgets everything the moment the process stops — exactly the problem Step 2 fixes with real storage. The function's docstring becomes the tool's description, which an MCP client hands straight to the model — Doc06's "a tool's description is really a prompt" idea, now over MCP instead of LangChain. **If not:** without `mcp.run()` the server never actually starts listening, and without the stderr-only logger, a stray `print()` would corrupt the protocol.

```
# server.py
# The MCPForge MCP server, Step 1: one Tool (add_note) over stdio, backed by
# an in-memory list. Run with `mcp dev server.py`. NEVER print() here or in
# anything it imports — on stdio, stdout is the protocol channel; all logging
# goes to stderr via get_logger.
from mcp.server.mcpserver import MCPServer

from src.config import load_config
from src.utils.logger import get_logger

config = load_config()
logger = get_logger(__name__)

mcp = MCPServer("notes-server")
notes: list = []

@mcp.tool()
def add_note(text):
    """Add a new note to the local notes store. Use this whenever the user
    wants to save or jot down a piece of text. Returns a short
    confirmation once the note is stored."""
    notes.append(text)
    logger.info(f"Note added (in-memory, count={len(notes)})")
    return f"Note added. There are now {len(notes)} note(s)."

if __name__ == "__main__":
    logger.info("Starting notes-server over stdio")
    mcp.run(transport="stdio")
```

Run it: `mcp dev server.py` opens the MCP Inspector; call `add_note` with `{"text": "Buy milk"}` and you get `"Note added. There are now 1 note(s)."`. Call it again in the same session → `"...now 2 note(s)."`. Close and reopen (a fresh process) and the count resets to 0 — this version really is only in memory.

The Step 1 tests

Story — `tests/unit/test_config.py`: this checks the settings code with no server running and a made-up setting name, so your real `.env` is never touched. **If not:** a broken config would surface as a confusing error deep inside the server at startup, far from the actual cause.

```
# tests/unit/test_config.py
# Checks the settings code with no server. Each case prints OK or WRONG.
# It only changes settings inside this test run — your .env is never touched.
# Run from the repo root: python -m tests.unit.test_config
import os

from src.config import load_config, require_env
from src.exceptions import MissingConfigError

TEST_SETTING = "MCPFORGE_TEST_SETTING"


def test_missing_setting():
    os.environ.pop(TEST_SETTING, None)
    result = "WRONG"
    try:
        require_env(TEST_SETTING)
    except MissingConfigError:
        result = "OK"
    print(f"case 1 (missing setting) -> {result}")


def test_present_setting():
    os.environ[TEST_SETTING] = "hello"
    result = "OK"
    if require_env(TEST_SETTING) != "hello":
        result = "WRONG"
    print(f"case 2 (present setting)  -> {result}")


def test_config_loads_defaults():
    os.environ.pop("NOTES_DB_PATH", None)
    config = load_config()
    result = "OK"
```

```
    if config.notes_db_path != "notes.db":
        result = "WRONG"
    print(f"case 3 (config defaults)  -> {result}")


if __name__ == "__main__":
    test_missing_setting()
    test_present_setting()
    test_config_loads_defaults()
```

Story — `tests/integration/test_server.py`: an MCP server can only really be exercised through a client, so this test *is* a tiny client itself — it launches `server.py` as a subprocess over stdio, lists the tools, and calls `add_note` for real. Later steps add their own cases to this same growing file. **If not:** "the server works" would mean "the Inspector showed something once by hand," with nothing automated to re-run. This proves the exact code path a real client actually uses.

```
# tests/integration/test_server.py
# Launches server.py over stdio with a real MCP client and exercises what it
# exposes. Each case prints OK or WRONG. Needs the mcp SDK; no API key (this
# server calls no model). Run from the repo root:
#   python -m tests.integration.test_server
import asyncio

from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client


def show(case, passed):
    result = "OK"
    if not passed:
        result = "WRONG"
    print(f"{case} -> {result}")


async def main():
    server_params = StdioServerParameters(command="python", args=["server.py"])
    async with stdio_client(server_params) as (read, write):
        async with ClientSession(read, write) as session:
            await session.initialize()

            tools = await session.list_tools()
            names = []
            for t in tools.tools:
                names.append(t.name)
            show("case 1 (tools listed)", "add_note" in names)
```

```
            added = await session.call_tool("add_note", {"text": "Buy milk"})
            show("case 2 (add_note works)", "1 note" in added.content[0].text)


if __name__ == "__main__":
    asyncio.run(main())
```

Create the empty folders `tests/unit/` and `tests/integration/` too — tests run as modules from the repo root, so they need no `__init__.py`.

Run the tests (from the repo root):

```
python -m tests.unit.test_config
python -m tests.integration.test_server
```

Expected output:

```
case 1 (missing setting) -> OK
case 2 (present setting)  -> OK
case 3 (config defaults)  -> OK
```

```
case 1 (tools listed) -> OK
case 2 (add_note works) -> OK
```